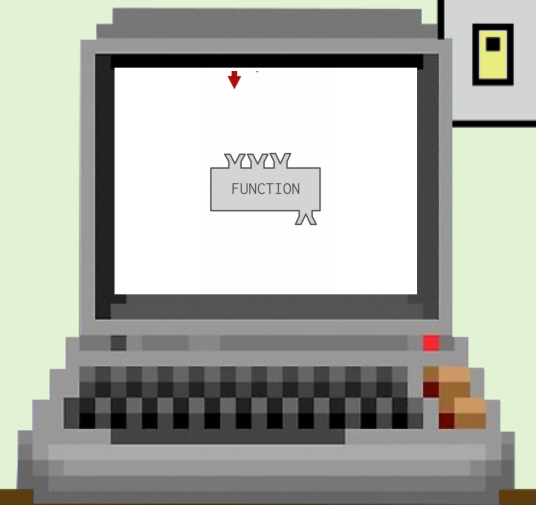


Funções de Alta Ordem

Programação Funcional



Funções de Alta Ordem

- Funções são “elementos de primeira categoria”
 - Podem ser armazenadas, passadas como parâmetro, retornadas por uma função
 - Ex: $\text{twice } f \ x = f (f \ x)$
- Exemplo

```
dobra :: Int -> Int
dobra n = 2*n
aplicaLista :: (Int->Int) -> [Int]->[Int]
aplicaLista f l = [f e | e<-l]
```

```
aplicaLista dobra [1,2,3]
```



Funções de Alta Ordem

- Função map
 - Recebe uma função e uma lista como argumentos
 - Aplica a função a cada elemento da lista
- Exemplo
 - `map sqrt [0,1,4,9]`
- Qual a definição da função map?



Funções de Alta Ordem

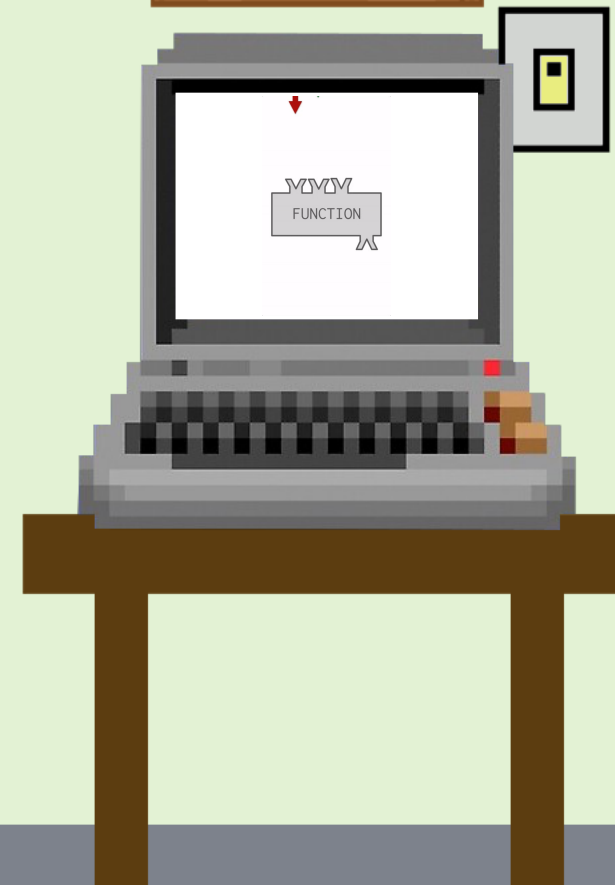
- Função map
 - Recebe uma função e uma lista como argumentos
 - Aplica a função a cada elemento da lista
- Exemplo
 - `map sqrt [0,1,4,9]`
- Qual a definição da função map?

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```



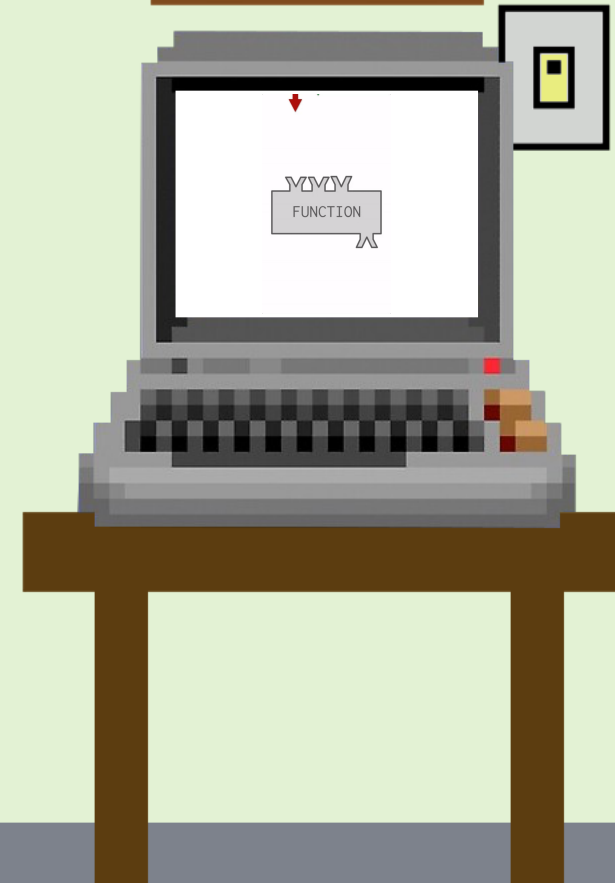
Funções de Alta Ordem

- Função filter
 - Escolhe dentre os elementos de uma lista, aqueles que têm uma determinada propriedade
 - Recebe uma função e uma lista
 - Retorna a lista filtrada
- Exemplo
 - `filter ehPar [2,3,4] = [2,4]`
 - `filter isDigit "A186 B70"`



Funções de Alta Ordem

- Funções foldr e foldl
 - Aplica uma função aos elementos de uma lista
 - Retorna o resultado da aplicação sucessiva
 - Reduz uma lista aplicando a função pela esquerda ou pela direita
- Funções foldr1 e foldl1
 - Semelhante a foldr e foldr
 - Não aceitam listas vazias



Funções de Alta Ordem

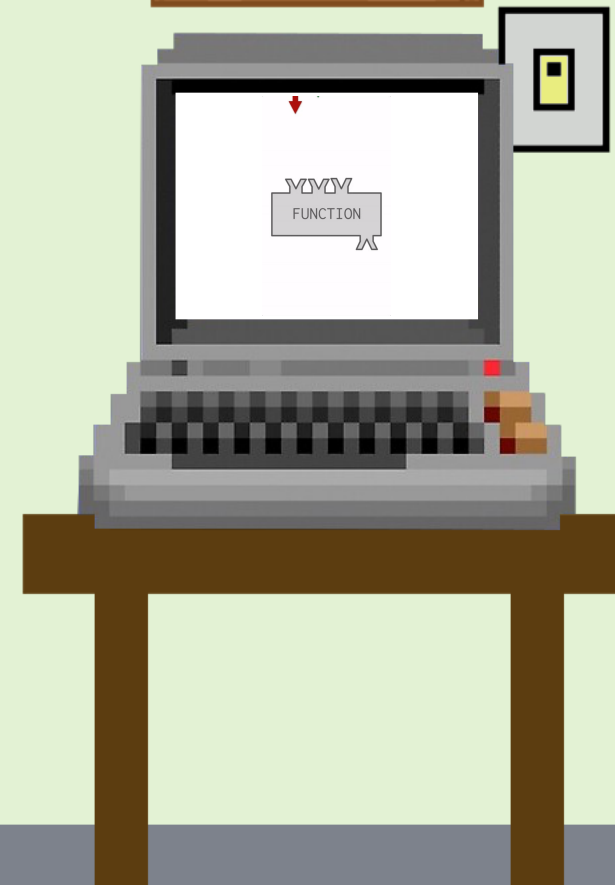
```
foldr (+) 0 [1..3]
1 + (foldr (+) 0 [2..3])
1 + (2 + (foldr (+) 0 [3]))
1 + (2 + (3 + (foldr (+) 0 [])))
```

```
foldl (+) 0 [1..3]
foldl (+) z1 [2..3] (z1 = 0+1)
foldl (+) z2 [3] (z2 = z1+2)
foldl (+) z3 [] (z3 = z2+3)
```



Composição de Funções

- A expressão $f(g(x))$, normalmente, é escrita pelos matemáticos como $(f.g)(x)$
 - Onde o $.$ (ponto) é o operador de composição
- O operador de composição possui como argumentos duas funções e o resultado é também uma função
- Exemplo
 - `sqrt . abs`
 - `(sqrt . abs) (-9)`



Composição de Funções

- Nem todo par de funções pode ser composto
 - O tipo da saída da função g tem de ser o mesmo tipo da entrada da função f
 - O tipo de $.$ é: $(.) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$
- A composição é associativa
 - $f.(g.h) = (f.g).h$
 - Deve ser interpretado como "faça h , faça g e finalmente faça f "



Aplicação Parcial

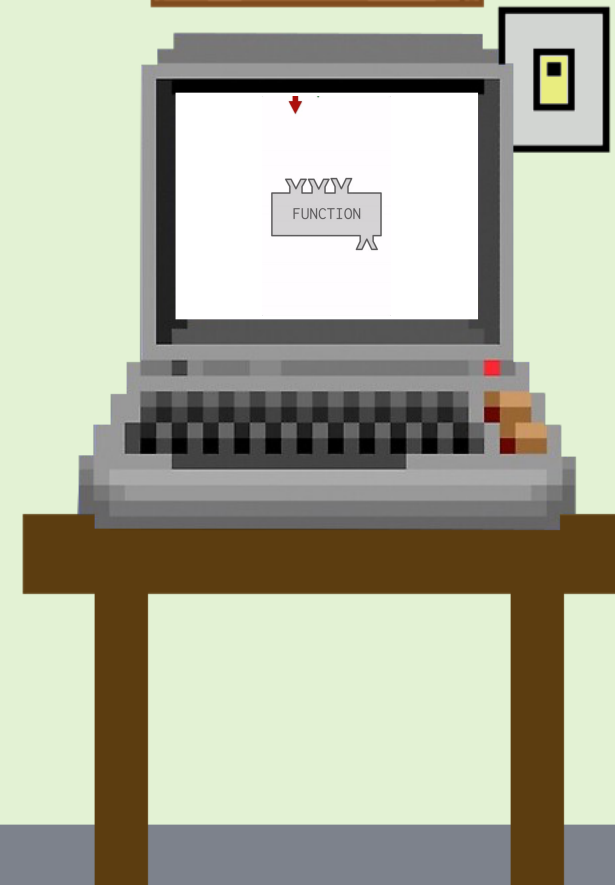
- Permite a construção de funções a partir da aplicação de um função a menos argumentos do que ela precisa
- Baseado na curificação
- Exemplo

```
dobraLista :: [Int] -> [Int]  
dobraLista = map (*2)
```



Seção de Operadores

- Decorrência direta das aplicações parciais
- São aplicações parciais, normalmente relacionados a operadores aritméticos
- Nas aplicações são colocadas entre parênteses
 - `(+2)` adiciona algum argumento a 2
 - `(2+)` adiciona 2 a algum argumento
 - `(>2)` retorna True se um inteiro for maior que 2
 - `(3:)` coloca o inteiro 3 na cabeça de uma lista
 - `(++"n")` coloca o "n" ao final de uma string



Exercícios

- Defina funções que tomem uma lista de inteiros L e
 - Retorne a lista dos quadrados dos elementos de L
 - Retorne a soma dos quadrados dos elementos de L
 - Verifique se todos os elementos da lista são positivos
- Escreva funções que calculem
 - O valor mínimo de uma função aplicada a uma lista de inteiros de 0 a n
 - Teste se o resultado de uma função f aplicada sobre as entradas de 0 a n são todas iguais
 - Teste se todos os valores de f aplicada às entradas de 0 a n são maiores ou iguais a 0
 - Teste se todos os valores de f aplicada às entradas de 0 a n estão em ordem crescente

